
superpowers + Compound Engineering 기반의 Claude Code 워크플로우

컨텍스트 · 설계 · 지식 · 토큰 — 네 가지 고민을 푸는 7단계 파이프라인

PART 01 · 왜

오랜 고민

Claude Code를 오래 쓰며 마주한 네 가지 벽

I 네 가지 고민

- 컨텍스트 관리 — 여러 파일을 동시에 들쑤시면 모델이 초기 결정을 잊고, **컨텍스트 부패**로 길을 잃는다
- 설계 · 계획의 구체화 — 모든 엣지 케이스를 직접 지시하는 대신, 모델이 **스스로 따져 묻게** 만들고 싶다
- 세션 지식의 관리 — **CLAUDE.md**에 규칙은 담아도, **경험은 담기지 않는다**
- 효율적 토큰 활용 — 한 줄 고치려 파일 전체를 다시 읽는 **낭비**를 줄인다

Ⅰ 느슨해서는 안 됐다

작업 순서 · 입출력 게이트 · 산출물 위치 · 토큰 통제까지

긴밀하게 엮인 하나의 워크플로우로

이 글은 그 일곱 단계를 처음부터 끝까지 다룬다

PART 02 · 도구

두 개의 도구

같은 영역, 다른 철학

I superpowers vs Compound Engineering

superpowers

한 세션 안에서, 엄격한 절차

모델의 기본 행동을 덮어쓰고 정해진 절차를 강제한다. “한 모델이 한 세션 안에서 어떻게 규율을 지키며 일하는가”에 답한다.

Compound Engineering

여러 에이전트, 시간에 따른 누적

서브에이전트가 산출물 파일로 협력하고, 세션의 지식이 복리로 쌓인다. “세션의 지식을 어떻게 다음 세션에 건넬 것인가”에 답한다.

I superpowers — 강제 절차 레이어

모델의 기본 행동을 **오버라이드**하는 강제 절차 시스템이다. (Jesse Vincent 제작 · 現 Anthropic 공식 마켓플레이스 등재)

- 핵심 규칙 — “적용될 스킬이 **1%라도** 의심되면 즉시 호출한다”
- 맥락을 읽고 곧장 구현으로 들어가던 패턴을 절차 레이어로 차단
- 5.1.0 기준 **14개 스킬** 제공

I superpowers 14개 스킬

스킬	역할	본 글 등급
using-superpowers	세션 시작 시 다른 스킬의 트리거 규칙을 설치하는 메타 게이트	보통
brainstorming	아이디어를 스펙으로 다듬음 · 95% confidence opener 추가	깊게
writing-plans	스펙을 task 단위 구현 계획으로 변환	보통
executing-plans	Plan을 task·step 순서로 실행 + 매 step 검증	보통
subagent-driven-development	Task당 서브에이전트 할당 + 작업 후 리뷰	언급
test-driven-development	RED → GREEN → REFACTOR 강제	깊게
systematic-debugging	가설 수립·검증을 절차로 묶어 디버깅	보통
verification-before-completion	완료 선언 전 type · lint · test 일괄 검증	깊게
finishing-a-development-branch	작업 마무리 · 커밋 · 푸시 · PR 작성	깊게
requesting / receiving-code-review	코드 리뷰 요청하고 수용	언급
dispatching-parallel-agents / git-worktrees	병렬 서브에이전트 할당 · Git Worktree 활용	언급
writing-skills	새 스킬 자체를 만드는 메타 스킬	언급

I 무엇이 다른가 — 엄격 vs 유연

RIGID · 엄격

체크리스트를 그대로

`test-driven-development`, `systematic-debugging` — 절차를 건너뛸 수 없다.

FLEXIBLE · 유연

원칙을 상황에 맞춰

`brainstorming` — 상황마다 다르게 적용한다. 각 스킬이 자신의 유형을 명시해 모델이 넘겨짚지 않게 한다.

I 95% Confidence Opener

“

지금 만들려는 것에 대해 1~2문장으로 설명해 주세요. 저는 95% 확신이 생길 때까지 질문을 던지겠습니다.
표면적으로 원하는 것이 아니라 **진짜로 필요한 것**을 짚기 위해서입니다.

— brainstorming · 설계 단계 게이트

I 방향을 잡는 5가지 **rigor probe**

- **Evidence** — 말한 want가 아니라 실제로 한 행동(시간·비용·우회책)이 있나
- **Specificity** — 구체적 수혜자는 누구이고 그에게 무엇이 바뀌나
- **Counterfactual** — 지금은 어떻게 하나, 안 만들면 무엇이 바뀌나
- **Attachment** — 같은 가치를 주는 가장 작은 형태는 무엇인가
- **Durability** — 가까운 변화에 이 가정이 견디나

제품화 작업에만 적용 — 리팩토링·문서엔 미적용. 출처: CE **/ce-brainstorm**의 Product Pressure Test.

I 어떻게 적용했나 — `~/ .claude/rules/`

... ~/ .CLAUDE/RULES/

~/ .claude/

└─ rules/

└─ boundaries.md	# 3-Tier 경계 (Auto / Ask / Never)
└─ karpathy-principles.md	# 코딩 원칙 — YAGNI, 외과적 변경
└─ hybrid-workflow.md	# 7단계 파이프라인 운영 규칙
└─ git-workflow.md	# 커밋 · 브랜치 · PR 포맷
└─ security.md	# 보안 체크리스트 · 시크릿 관리

세션 시작 시 자동 로드되어 에이전트가 항상 참조한다. **경계(boundaries)**와 **원칙(karpathy)**이 행동의 두 축이다.

I Compound Engineering — 협력과 누적

서브에이전트 병렬화 + 학습 누적 시스템. Every의 Kieran Klaassen이 만들었다.

산출물 파일로 협력

여러 서브에이전트가 독립 컨텍스트에서 일하고, 결과를 파일로 주고받는다.

시간에 따른 복리

세션에서 얻은 지식이 학습으로 누적되어, 작업을 거듭할수록 두터워진다.

I CE 핵심 5요소

명령어 / 에이전트	역할
/ce-plan	병렬 서브에이전트로 리서치 후 계획 생성 (CodeGraph·graphify 결합)
ce-learnings-researcher	docs/solutions/ 의 과거 학습을 frontmatter로 회수
/ce-work	계획을 task 단위로 실행 — 내장 워크트리 · 병렬 안전
/ce-code-review	6개 이상 리뷰어 페르소나 앙상블
/ce-compound	작업 학습을 docs/solutions/ 로 저장 — CE의 핵심

I 학습 루프 — 복리 효과

STEP 1 · 저장

compound

`/ce-compound` 가 헤드리스로 `docs/solutions/` 에
저장 (frontmatter 자동 분류)



STEP 2 · 회수

researcher

다음 `/ce-plan` 에서 `ce-learnings-`
`researcher` 가 관련 학습 회수



STEP 3 · 복리

누적

작업이 쌓일수록 같은 실수를 반복할 확률이 낮아진다

I 왜 학습 누적인가

CLAUDE.md에 규칙은 담을 수 있다



하지만 경험은 담기지 않는다

그래서 매 작업의 학습을 **docs/solutions/**에 누적한다 — CE의 복리 효과

PART 03 · 선택

접치는 영역

작업 단위마다 무엇을 고를까

I 선택의 두 원칙

- 01 각 도구의 철학을 이해한다 — 세션 내 규율은 superpowers, 협력·누적은 Compound Engineering
- 02 단일 책임 + 산출물 파일 통신 — 작업 선별~PR을 한 스킬로 묶는 일체형을 피하고, 단계별 단독 스킬에 초점

- 작업 단위 ① · 브레인스토밍

I 브레인스토밍 → superpowers

- CE에도 있고(`/ce-ideate` · `/ce-brainstorm`), Product Pressure Test는 더 엄격하기도 하다
- 그러나 내 작업은 개발만이 아님 — 대규모 리팩토링·광범위 문서에선 제품화 질문이 **헛돈다**
- superpowers는 작업 특성에 **중립적**
- 결론: 도구는 superpowers, 질문 방법론(rigor probe)만 `/ce-brainstorm`에서 흡수

- 작업 단위 ② · 계획

I 계획 → CE `/ce-plan`

`writing-plans` (단일 모델·단일 컨텍스트)보다 정교하다.

- 병렬 리서치 — `ce-repo-research-analyst` (스택·패턴) + `ce-learnings-researcher` (과거 학습) + 조건부 외부 리서치
- 각 서브에이전트는 독립 컨텍스트로 조사하고 요약만 메인에 반환 → 메인 컨텍스트는 가볍게
- 구현 단위마다 ID + 4 테스트 범주(정상·엣지·에러·통합) 강제 → `/ce-work` 로 바로 실행 가능

- 작업 단위 ③ · 실행

I 실행 → CE /ce-work

`subagent-driven-development` 는 새 서브에이전트를 띄우되 **병렬을 원천 금지**한다.

- **병렬 안전성 검사** — 각 작업의 `Files:` 로 파일-작업 매핑 → 교집합 사전 검증 → 워크트리 격리 가능하면 병렬, 아니면 순차
- **3중 안전망** — ① 테스트 탐색 ② 4 시나리오 완전성 ③ 전역 5문항(무엇 실행 / 실제 체인 / 실패 시 고아 / 인터페이스 노출 / 레이어 에러 전략)

- 작업 단위 ④ · 코드 리뷰

I 코드 리뷰 → CE `/ce-code-review`

superpowers는 범용 리뷰어 하나. CE는 **전문 페르소나 앙상블**이다.

- **항상** — correctness · maintainability · testing · project-standards (+ CE 2개: agent-native · learnings)
- **조건부** — security · performance · data-migration · api-contract ...
- 여럿이 **독립적으로** 같은 이슈 → 중요도 높음 / 다른 이슈 → 다각도 커버

I 디버깅 → **superpowers** systematic-debugging

철칙 — “근본 원인 조사 없이는 어떤 수정도 없다”

- 절차는 4단계로 단순 — 근본 원인 → 패턴 분석 → 가설 → 구현
- Red Flags + ‘흔한 합리화’ 표(8항목)로 모델의 **내적 발화**를 문장 단위로 차단
- **/ce-debug** (5단계·이슈 자동 fetch)도 정교하나, 단일 책임 + 합리화 차단 때문에 superpowers 선택

I 검증 · 완료 · 커밋 → **superpowers**

“수정 완료”가 검증 기반인지 추론 기반인지부터 판별해야 한다.

- `verification-before-completion` — “검증 명령을 실행 안 했다면 통과를 주장할 수 없다”. `ty` · `ruff` · `pytest` 직접 확인
- `finishing-a-development-branch` — PR 병합 방식을 사람이 선택
- 일체형 `/ce-commit-push-pr` 대신 → 사람 게이트 2단으로 리스크 최소화

- 작업 단위 ⑦ · 학습 누적

I 학습 누적 → CE 단독 `/ce-compound`

superpowers엔 이 단계가 전혀 없다.

- 3개 서브에이전트로 세션 학습을 `docs/solutions/<problem>.md` 로 추출 (frontmatter 자동 분류)
- 다음 `/ce-plan` 에서 `ce-learnings-researcher` 가 회수 → 시간 흐름의 복리
- 작업이 늘수록 자동으로 두터워지는 지식 — 시니어가 후배에게 “우린 이렇게 해결했어”를 건네는 것과 같다

I 한눈에 보는 단계별 도구

작업 단위	superpowers	Compound Engineering	선택
브레인스토밍	brainstorming	ce-brainstorm · ce-ideate	superpowers
plan 작성	writing-plans	/ce-plan	CE
plan 실행	executing-plans	/ce-work	CE
코드 리뷰	requesting-code-review	/ce-code-review	CE
디버깅	systematic-debugging	/ce-debug	superpowers
검증 · 완료 · 커밋	verification + finishing	/ce-commit-push-pr	superpowers
학습 누적	—	/ce-compound	CE 단독

PART 04 · 워크플로우

실제 작업 흐름

7단계를 네 개의 페이지즈로

I 네 페이지 한눈에

PHASE 1

설계 · Spec

brainstorming
→ docs/superpowers/specs/



PHASE 2

계획 · Plan

/ce-plan + plannotator
→ docs/plans/



PHASE 3

구현 · Build

TDD + /ce-work
+ /ce-code-review



PHASE 4

마무리 · 학습

verification + /ce-compound
+ finishing-branch

각 페이지 사이마다 **/clear**로 컨텍스트를 초기화한다

I 설계 — 모호한 아이디어를 **스펙**으로

- superpowers **brainstorming** + 95% opener → **docs/superpowers/specs/**
- 스펙은 구현 세부가 아니라 **의사결정**을 담는다 — 아키텍처 · 검토한 옵션 · 진행 가정
- 추론 비중이 커서 Opus 4.8 xhigh 사용, 끝나면 **/clear**로 컨텍스트 초기화

I 계획 — 실행 가능한 단위로

- Plan Mode 진입 → `/ce-plan`: CodeGraph(`explore` · `callers` · `impact`) + `ce-learnings-researcher` (solutions ≥ 3 개)
- 서브에이전트가 별도 컨텍스트로 리서치 → 요약만 반환, 메인은 가볍게
- `ExitPlanMode` → **Plannotator** Hook 자동 발동 → 줄 단위 어노테이션·승인 → `docs/plans/`

I 구현 — 잘 짜인 계획을 그대로

- 이미 정련된 계획을 읽고 구현 → 과한 추론 불필요, **Sonnet 4.6 high**
- `test-driven-development` RED → GREEN → REFACTOR (트리비얼 변경은 면제)
- `/ce-work <plan>` — 편집 전 `codegraph_impact` 로 blast radius 확인
- `/ce-code-review` — 아무리 잘 짜도 수정거리 하나는 반드시 나온다

I 마무리 · 학습 — 흔적을 남긴다

- `verification-before-completion` — `ty` · `ruff` · `pytest`, 하나라도 실패 시 완료 선언 불가
- `/ce-compound mode:headless` → `docs/solutions/` (Tier 0/1 자동 반영은 금지)
- `finishing-a-development-branch` — 한국어 커밋 포맷으로 의미 있게 마무리

PART 05 · 토큰 효율

고민 4 · 효율적 토큰 활용

보는 범위를 좁히고, **출력을 압축한다**

I 세 가지 보조 인프라

도구	메커니즘	무엇을 재나	절감
CodeGraph	보는 범위 좁힘 (심볼 그래프)	질문 1건당 cost · tool call	툴 호출 58% ↓
graphify	스코프 서브그래프	쿼리당 토큰	쿼리당 71.5×
RTK	명령어 출력 압축	명령 1건당 출력 토큰	명령별 60–90%

도구가 항상 정답은 아니다 — 더 맞는 게 있으면 바꾼다. (이 글을 쓰며 Serena MCP → CodeGraph로 교체했다.)

I CodeGraph — 심볼만 콕 집어서

● ● ● CODEGRAPH_IMPACT

시그니처 변경 전 영향 범위 확인

`codegraph_impact("AuthService.verify")`

→ `api/login.py` : 직접 호출

→ `api/refresh.py` : 직접 호출

→ `mw/session.py` : 동적 디스패치

- `codegraph_explore` — NL/심볼 → 소스 (Read 대체)
- `codegraph_callers` — 누가 이 함수를 호출하나
- `codegraph_impact` — 수정 **전에** 영향 범위, grep이 못 따라가는 동적 디스패치까지

I CodeGraph — 비용 · 토큰 절감

코드베이스	언어 · 규모	비용	토큰	시간	틀 호출
VS Code	TypeScript · ~10k	18%	64%	11%	81%
Excalidraw	TypeScript · ~640	—	25%	27%	40%
Django	Python · ~3k	8%	60%	13%	77%
Tokio	Rust · ~790	—	38%	18%	57%
OkHttp	Java · ~645	25%	54%	31%	50%
Gin	Go · ~110	19%	23%	24%	44%
Alamofire	Swift · ~110	40%	64%	33%	58%

graphify — 무엇이든 그래프로

- 코드뿐 아니라 문서 · 논문 · 이미지까지 그래프로 변환 → `graphify-out/`
- `graphify query "결제 모듈은 어디서 인증을 호출하나?"` — 자연어 문장을 받는다
- 답으로 BFS/DFS 탐색의 **스코프된 서브그래프**만 반환 → 보는 영역을 질문에 맞게 좁힘
- 대안(`code-review-graph` 등) 대비 강점: 코드만이 아니라 **문서**를 다룬다

I RTK — 흐르는 출력을 압축

●●● RTK GAIN

RTK Token Savings (Global Scope)

Total commands: 5,861

Input tokens: 5.9M

Output tokens: 3.0M

Tokens saved: 3.0M (49.6%)

#	Command	Count	Saved	Avg%
1.	rtk read	425	2.2M	22.0%
2.	rtk grep	881	194.6K	11.0%
8.	rtk git commit	259	24.8K	96.1%

Hook 기반으로 `git status`·`ls`가 자동으로 `rtk` 버전으로 치환된다. 입력 5.9M 중 약 50% 절감, `git commit`은 96%까지.

PART 06 · 하 네 스

모델이 못 하는 일

지능으로 못 메우는 빈틈을 하네스로

I 모델이 스스로 못 하는 일

모델의 지능으로는 메울 수 없는 빈틈이 있다 — 이 빈틈은 하네스(Hook)가 대신 강제하고 보완한다.

- 세션 도중 모델 · 추론 수준을 바꾸는 일
- 단계가 바뀔 때마다 컨텍스트를 초기화하고 다음 단계의 규칙을 불러오는 일
- 명령어의 장황한 출력을 알아서 압축하는 일

I 세 개의 핵심 Hook

이벤트	matcher	실행 대상	역할
PostToolUse	Skill	workflow-stage-inject.sh	스킬 호출 직후 단계 지침 주입 · 규율
PostToolUse	ExitPlanMode	인 라 인 스 크 립 트	plan 승인 직후 확정 · 인계 강제 · 규율
PreToolUse	Bash	rtk-hook.sh	Bash 실행 직전 출력 압축 라우팅 · 압축

PreToolUse = 실행 **전** 입력 자체를 바꾼다(압축) · **PostToolUse** = 실행 **후** 컨텍스트를 주입한다(규율).

I 단계 지침을 주입한다 — workflow-stage-inject.sh

● ● ● WORKFLOW-STAGE-INJECT.SH

```
#!/usr/bin/env bash
input=$(cat)
skill=$(echo "$input" | grep -oP '"skill": "\K[^\"]*')

emit() { printf '{ ...additionalContext: "%s" }' "$1"; }

case "$skill" in
    *brainstorming) emit "첫 turn 95% opener 강제 · 5렌즈 probe" ;;
    *ce-work)       emit "Sonnet·high 권장 → /model 전환 안내" ;;
    *verification-*) emit "검증 명령 실제 실행 후 완료 선언" ;;
esac
exit 0 # 목록에 없는 스킬은 조용히 통과
```

- HOOK ① · 무엇을 주입하나

I 권장 모델부터 단계 규칙까지

- 호출된 스킬명을 7단계 파이프라인 목록과 대조 → 단계 지침을 `additionalContext`로 주입
- 주입 내용: 권장 모델 · 추론 수준 · `/clear` 시점 · 단계 고유 규칙
- 예: `brainstorming` → 95% opener / `ce-work` → Sonnet 전환 안내 / `verification` → 검증 명령 실행
- 핵심: 강제가 아니라 안내(넛지) — 세션 모델·추론은 모델이 스스로 못 바꾸므로

I Plan 확정을 강제한다 — Plannotator 승인

- Plannotator 승인 게이트 위에 Hook을 얹는다 — 승인 시에만 발동 (어노테이션·수정 요청은 deny → 미발동)
- ① 계획을 짰 세션에서 **곧장 인라인 구현 금지** — 컨텍스트 부패 방지
- ② 승인된 계획을 `~/.claude/plans/` → 프로젝트 `docs/plans/` 로 **승격** (프론트매터 보존)
- ③ `/clear` 후 새 세션에서 `/ce-work <경로>` 로 구현(Sonnet high) 시작 안내

I `uv run` 사각지대를 메운다 — `rtk-hook.sh`

●●● RTK-HOOK.SH

```
# uv run <tool> → uv run rtk <tool>
case "$CMD" in
  "uv run pytest"*) new="uv run rtk pytest..." ;;
  "uv run ruff"*)   new="uv run rtk ruff..." ;;
  "uv run mypy"*)   new="uv run rtk mypy..." ;;
esac
# ty 는 RTK 미지원 → 그대로 통과
```

- RTK는 `uv run` 으로 감싼 명령을 못 잡는다 — 하필 TDD 필수(`pytest` · `ruff`)라 출력이 장황
- `uv run pytest ...` → `uv run rtk pytest ...`로 치환 (`pytest` · `ruff` · `mypy`)
- `ty` 는 RTK 미지원이라 통과 / 그 외 명령은 짝 혹 `rtk-rewrite.sh`가 담당

I 정리 — 빈틈은 하네스로

지능으로 못 메우는 빈틈은 **하네스**가 메운다

강제가 아니라, 잊지 않도록 안내한다

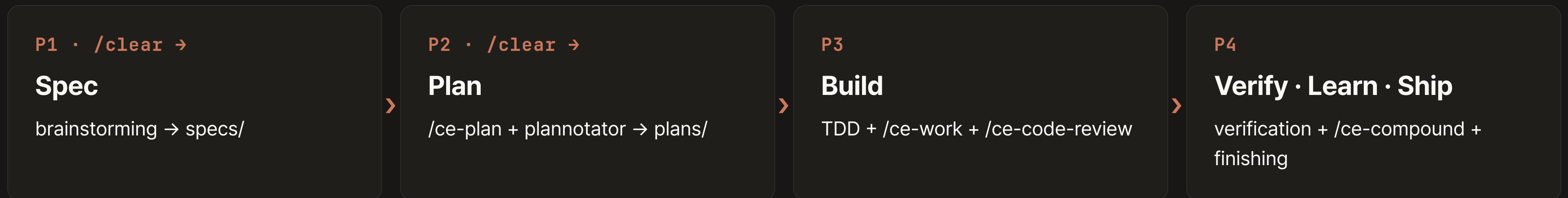
단계가 바뀔 때마다 하네스가 작은 체크포인트를 건넨다

PART 07 · 정리

한눈에 다시

파이프라인과 메모리 시스템

I 7단계 파이프라인 한눈에



각 단계는 새 컨텍스트 + 입력 파일만 받는다 → 컨텍스트 폭발 방지. P4 → P2 학습 루프로 닫힌다.

I 메모리 · 문서 시스템

Tier	위치	책임	변경 주체
0	~/.claude/CLAUDE.md · rules/	글로벌 행동 규칙 · 게이팅	사용자 수동만
1	~/.claude/.../memory/	user · feedback · project · reference	모델(발화 한정)
2	docs/superpowers/specs · plans	의사결정 — spec · plan	모델 + 승인 게이트
3	docs/solutions/	학습 누적 (compound + researcher)	모델 자동(headless)
4	graphify-out/	시각화	사용자/모델 호출 시
5	TODO.md	후속 작업 추적	양쪽

핵심 규칙: Tier 0/1로의 자동 반영은 금지. 가치 있는 학습은 사람이 손으로 승격한다.

I 나가며

내 작업을 **효율적이면서 품질을 높일 수 있는 것이 있다면**

언제든 고민해 봐야 한다 — 이 워크플로우와 하네스는 늘 바뀔 수 있다

감사합니다 😊

Part 2에서 실제 작업으로 이어집니다